WINDOWS 11 ACTION RUNBOOK

LG Monitor Companion-Software Cleanup and Recurrence Mitigation

11
PRO

Windows 11 Pro Edition
Preflight -> confirm exact LG package -> remove -> restart -> verify
Optional recurrence controls appear only when this edition supports the tested path

Author: Amyashi | v1.0.0 | Reviewed 29 July 2026

Original observed case: Windows 11 Pro; revalidate the package and delivery path locally.

Independent, unofficial and unaffiliated; not endorsed by LG or Microsoft; not support, certification or a guarantee. Guide materials: CC BY-NC-ND 4.0; see LICENSE.md.

Core path

0. Preflight and capture state
1. Apply this edition's metadata control
2. Remove only the exact LG package
2A. Decide about LG OnScreen Control separately
3. Restart once and verify
4. Stop when every core result passes

Recurrence path

Use AppLocker only after the exact tested package actually returns. Leave that returned package installed, reconfirm its full identity, start in **Audit only**, and enforce only after every gate passes.

READ BEFORE USE

Use only on a personally administered PC or with the responsible administrator's approval. All-user package removal affects every current user and future-user provisioning and requires reinstallation to undo. Confirm that no firmware update is running and that a working remote, physical control path or accepted loss of software-only controls exists.

DOCUMENTED EXAMPLE AND ELIGIBILITY

This procedure was developed from an LG 45GR95QE-B case involving the Microsoft Store product **LG Monitor App Installer** (Store ID `9PM9N6F47JB8`, package `LGElectronics.LGMonitorApp`) and the separate LG OnScreen Control desktop application. Other LG monitor models can use different software or delivery paths. Confirm the exact model's official LG support page and stop if the exact package is absent, its Store publisher is not LG Electronics Inc., or a different LG-named package appears. Never substitute another package name into these commands. [25] [23]

1. Step 0 - preflight and preserve the before-state

Complete this page before changing Windows. Keep the same elevated PowerShell window open through the cleanup step.

PHYSICAL AND RECOVERY CHECK

Confirm that **no LG monitor firmware update is running**. Record the exact monitor model and open its official LG support page. Confirm a working remote or physical control path, or deliberately accept losing software-only controls. Policy backups do not reinstall removed software. [23] [24]

A - CONFIRM EDITION, ADMINISTRATOR AND DOMAIN STATE

```
# Run in Windows PowerShell 5.1 as Administrator.
$ErrorActionPreference = 'Stop'
$ExpectedEdition = 'Windows 11 Pro'
$Os = Get-CimInstance Win32_OperatingSystem
$Computer = Get-CimInstance Win32_ComputerSystem
$Identity = [Security.Principal.WindowsIdentity]::GetCurrent()
$Principal = [Security.Principal.WindowsPrincipal]$Identity
$IsAdmin = $Principal.IsInRole(
    [Security.Principal.WindowsBuiltInRole]::Administrator)
[pscustomobject]@{
    WindowsEdition = $Os.Caption
    Version = $Os.Version
    Build = $Os.BuildNumber
    Administrator = $IsAdmin
    DomainJoined = $Computer.PartOfDomain
}
if (-not $IsAdmin) { throw 'STOP: reopen PowerShell as Administrator.' }
if ($Computer.PartOfDomain) { throw 'STOP: use the managed-device workflow.' }
if ($Os.Caption -notmatch [regex]::Escape($ExpectedEdition)) {
    throw "STOP: this is the $ExpectedEdition guide; detected $($Os.Caption)."
}
$RunId = '{0}-{1}' -f (Get-Date -Format 'yyyyMMdd-HHmmss'),
    ([guid]::NewGuid().ToString('N').Substring(0,8))
$RunDir = Join-Path ([Environment]::GetFolderPath('Desktop')) "LG-Run-$RunId"
New-Item -ItemType Directory -Path $RunDir -ErrorAction Stop | Out-Null
$RunDir
```

B - CAPTURE THE EXACT CORE BEFORE-STATE

```
$Package = 'LGElectronics.LGMonitorApp'
$ExpectedPublisher = 'CN=4A5401D9-32C2-486E-A8CB-EFECB124AF89'
$ExpectedPublisherId = 'cfnzzhwkr8z5w'
$ExpectedFamily = "${Package}_${ExpectedPublisherId}"
$Types = 'Main','Framework','Resource','Bundle','Optional'
$MetaPath = 'HKLM:\SOFTWARE\Policies\Microsoft\Windows\Device Metadata'
$MetaName = 'PreventDeviceMetadataFromNetwork'
$Meta = [ordered]@{ KeyExisted = Test-Path -LiteralPath $MetaPath
                    ValueExisted = $false; Value = $null; Kind = $null }
if ($Meta.KeyExisted) {
    $Key = Get-Item -LiteralPath $MetaPath
    $Props = Get-ItemProperty -LiteralPath $MetaPath
    $Meta.ValueExisted = $Props.PSObject.Properties.Name -contains $MetaName
    if ($Meta.ValueExisted) {
        $Meta.Value = $Props.$MetaName
        $Meta.Kind = [string]$Key.GetValueKind($MetaName)
    }
}
[pscustomobject]$Meta | Export-Clixml "$RunDir\Metadata-Before.clixml"
Get-AppxPackage -AllUsers -Name $Package -PackageTypeFilter $Types |
    Export-Clixml "$RunDir\Registered-Before.clixml"
Get-AppxProvisionedPackage -Online |
    Where-Object DisplayName -eq $Package |
    Export-Clixml "$RunDir\Provisioned-Before.clixml"
Get-MpPreference | Select-Object PUAProtection |
    Export-Clixml "$RunDir\Defender-Before.clixml"
Get-ChildItem -LiteralPath $RunDir |
    Select-Object Name,Length,LastWriteTime
```

GO / STOP

Open every saved file, then copy the complete `LG-Run-...` folder to separate administrator-controlled storage. Record the current Local Group Policy state before changing it. Open **Settings > Accounts > Access work or school**. Continue only when the edition matches this guide, Administrator is True, DomainJoined is False, no organization manages the PC, the recovery copy opens, and the exact Store product and publisher have been confirmed. Unknown package or publisher identity means stop.

2. Step 1 - reduce metadata-associated app retrieval

Use the supported Local Group Policy setting, verify it, and record the exact undo.

DO THIS - WINDOWS 11 PRO

1. Open `gpedit.msc`.
2. Go to Computer Configuration > Administrative Templates > System > Device Installation.
3. Open **Prevent automatic download of applications associated with device metadata**.
4. Select **Enabled**, then Apply and OK.
5. Run `gpupdate /target:computer /force`.
6. Run the verification block. Expected policy value: **1**.

Undo: restore the exact Local Group Policy state recorded in Step 0, refresh policy, and verify again. [01]

VERIFY THE EFFECTIVE REGISTRY RESULT

```
$Path = 'HKLM:\SOFTWARE\Policies\Microsoft\Windows\Device Metadata'
$Name = 'PreventDeviceMetadataFromNetwork'
$Value = Get-ItemPropertyValue -Path $Path -Name $Name `
    -ErrorAction SilentlyContinue
[pscustomobject]@{
    WindowsEdition = (Get-CimInstance Win32_OperatingSystem).Caption
    MetadataPolicyValue = if ($null -eq $Value) { '(not set)' } else { $Value }
}
```

PASS RESULT	IF IT DOES NOT MATCH	RESTART?
`MetadataPolicyValue = 1`	Reopen Local Group Policy, confirm Enabled, refresh Group Policy and recheck. Stop if another authority reverses it.	No restart yet. Continue to package cleanup.

CONTROL BOUNDARY

This policy concerns applications associated with device metadata. Neither control is a universal block for Microsoft Store/manual acquisition, Windows Update drivers, OEM provisioning, Driver Package Container Metadata, driver-INF `AddSoftware` or conventional Win32 installers. Microsoft also marks the legacy device-metadata path as deprecated. [21] [26] [22]

3. Step 2 - remove the exact LG package

This Windows 11 Pro procedure targets only LGElectronics.LGMonitorApp. Zero packages at the start means already clean, not an error.

IDENTITY AND RECOVERY GATE

Before running removal, open Microsoft Store product `9PM9N6F47JB8` and confirm **LG Monitor App Installer**, publisher **LG Electronics Inc.** The command then displays the exact installed name, publisher, publisher ID, family and full package identity before asking for `REMOVE`. It fails closed unless the registered signer is `CN=4A5401D9-32C2-486E-A8CB-EFECB124AF89`, the publisher ID is `cfnzzhwkr8z5w` and the family is `LGElectronics.LGMonitorApp_cfnzzhwkr8z5w`. These are tested identity values, not a promise about a future LG release. Stop and revalidate if the Store reference or any installed identity differs, or if another LG package appears. This is an all-user action: it removes registrations for current users and provisioning for future users. Never substitute another package name, delete WindowsApps files manually, or reinstall an absent package merely to create a rule. [25] [09] [10]

A - REMOVE EXISTING REGISTRATIONS AND FUTURE-USER PROVISIONING

```
$Package = 'LGElectronics.LGMonitorApp'
$ExpectedPublisher = 'CN=4A5401D9-32C2-486E-A8CB-EFECB124AF89'
$ExpectedPublisherId = 'cfnzzhwkr8z5w'
$ExpectedFamily = "${Package}_${ExpectedPublisherId}"
$Types = 'Main','Framework','Resource','Bundle','Optional'
$Targets = @(Get-AppxPackage -AllUsers -Name $Package `
    -PackageTypeFilter $Types -ErrorAction Stop)
$Provisioned = @(Get-AppxProvisionedPackage -Online -ErrorAction Stop |
    Where-Object DisplayName -eq $Package)
$Targets | Select-Object Name,Publisher,PublisherId,PackageFamilyName,PackageFullName
$Provisioned | Select-Object DisplayName,PublisherId,PackageName
if ($Targets.Count -eq 0 -and $Provisioned.Count -eq 0) {
    'Already clean: no registered or provisioned target was found.'
} else {
    if (@($Targets | Where-Object {
        $_.Publisher -ne $ExpectedPublisher -or
        $_.PublisherId -ne $ExpectedPublisherId -or
        $_.PackageFamilyName -ne $ExpectedFamily
    }).Count -gt 0) {
        throw 'STOP: registered signer or package family is not the tested identity.'
    }
    if (@($Provisioned | Where-Object {
        $_.PublisherId -ne $ExpectedPublisherId
    }).Count -gt 0) {
        throw 'STOP: provisioned publisher ID is not the tested identity.'
    }
    $Confirm = Read-Host "Type REMOVE to remove only $Package from all users"
    if ($Confirm -cne 'REMOVE') { throw 'STOP: removal was not confirmed.' }
    if ($Targets.Count -gt 0) {
        $Bundles = @(Get-AppxPackage -AllUsers -Name $Package `
            -PackageTypeFilter Bundle -ErrorAction Stop)
        $Mains = @(Get-AppxPackage -AllUsers -Name $Package `
            -PackageTypeFilter Main -ErrorAction Stop)
        $Parents = if ($Bundles.Count -gt 0) { $Bundles } else { $Mains }
        if ($Parents.Count -eq 0) {
            throw 'STOP: child packages exist without a removable parent.'
        }
        $Parents | Sort-Object PackageFullName -Unique | ForEach-Object {
            Remove-AppxPackage -Package $_.PackageFullName -AllUsers `
                -ErrorAction Stop
        }
    }
    $Provisioned | ForEach-Object {
        Remove-AppxProvisionedPackage -Online -AllUsers `
            -PackageName $_.PackageName -ErrorAction Stop | Out-Null
    }
}
```

B - VERIFY BOTH STORES OF PACKAGE STATE

```
$Package = 'LGElectronics.LGMonitorApp'
$Types = 'Main','Framework','Resource','Bundle','Optional'
$Registered = @(Get-AppxPackage -AllUsers -Name $Package `
    -PackageTypeFilter $Types -ErrorAction Stop)
$Provisioned = @(Get-AppxProvisionedPackage -Online -ErrorAction Stop |
    Where-Object DisplayName -eq $Package)
[pscustomobject]@{
    RegisteredPackageCount = $Registered.Count
    ProvisionedPackageCount = $Provisioned.Count
}
```

STEP 2A - DECIDE ABOUT LG ONSCREEN CONTROL

Action: Keep OnScreen Control if you use it for firmware, presets or settings. Remove it only through **Settings > Apps > Installed apps**, only when no firmware update is running and another accepted control path works.

Expected result: the chosen remote, physical or software control path works and no firmware update was interrupted.

Restart: no separate restart now; continue to Step 3.

If it fails / undo: stop. Reinstall only the official model-specific LG version, then retest the functions you use. [24] [23]

PASS RESULT

Both counts are **0**. Registered removal clears existing users; provisioning removal clears the online image used for future users. Both checks are required. [10] [11] [12]

4. Step 3 - restart once, then verify

Restart Windows now. Sign in normally, wait for the desktop to settle, then run the block in an elevated Windows PowerShell 5.1 window.

POST-RESTART CORE VERIFICATION

```
$Package = 'LGElectronics.LGMonitorApp'
$Types = 'Main','Framework','Resource','Bundle','Optional'
$Edition = (Get-CimInstance Win32_OperatingSystem).Caption
$Registered = @(Get-AppxPackage -AllUsers -Name $Package `
    -PackageTypeFilter $Types -ErrorAction Stop)
$Provisioned = @(Get-AppxProvisionedPackage -Online -ErrorAction Stop |
    Where-Object DisplayName -eq $Package)
$Meta = Get-ItemPropertyValue `
    -Path 'HKLM:\SOFTWARE\Policies\Microsoft\Windows\Device Metadata' `
    -Name PreventDeviceMetadataFromNetwork -ErrorAction SilentlyContinue
[pscustomobject]@{
    WindowsEdition = $Edition
    RegisteredPackageCount = $Registered.Count
    ProvisionedPackageCount = $Provisioned.Count
    MetadataPolicyValue = if ($null -eq $Meta) { '(not set)' } else { $Meta }
    DefenderRealTime = (Get-MpComputerStatus).RealTimeProtectionEnabled
}
```

CHECK	WINDOWS 11 PRO PASS	IF IT DOES NOT MATCH
Registered package	0	Return to Section 3; display identity again and stop on any mismatch.
Provisioned package	0	Return to Section 3; do not delete WindowsApps files manually.
Metadata control	Policy value = 1	Return to Section 2 and restore or reapply through the same control surface.
Defender health	True when Microsoft Defender is the active antivirus	Identify the active antivirus authority before changing PUA settings.
Monitor baseline	Every recorded function actually used still works	Restore only the smallest changed layer; use Section 10.

Everything passed

Stop here. The core mitigation is complete. Do not enable AppLocker or change Defender without a documented requirement. Record the date and keep the recovery folder.

Something failed

Continue to the failure ladder on the next page. Preserve the exact output and time. Do not repeat removal blindly, add broad exclusions or delete application folders.

FUNCTIONAL MONITOR CHECK

Compare against the pre-change baseline for this exact LG model: native resolution, intended maximum refresh rate, adaptive sync if supported, HDR if supported and used, audio, USB hub or KVM, input switching, firmware access and the chosen remote/physical/software control path. A clean package count is not a pass if a used monitor function was unintentionally lost.

5. Use the failure ladder - do not skip ahead

Choose the row matching the observed result. The optional AppLocker path begins only after the exact tested package actually returns, not from uncertainty.

OBSERVED RESULT	NEXT ACTION	DECISION GATE
Registered or provisioned count is nonzero	Display package name, publisher, type and full name again. Close the app; confirm other users are signed out; rerun only the exact removal block.	Still nonzero: stop and inspect package state. Never delete WindowsApps manually.
The exact package returns later	Record the time, edition, package identity and observed delivery path. Reconfirm the Pro metadata policy value without assuming it caused the return.	If repeat prevention is required, leave the exact returned package installed long enough to use as its own signed AppLocker reference, then continue to Section 6.
A different LG-named package appears	Stop. Record its full name, family, publisher and delivery time.	Do not substitute its name into this guide. Use the exact monitor model's official LG support page and obtain product-specific guidance.
OnScreen Control was needed	Use the official LG support source to reinstall the intended version.	Recheck firmware/settings access and the full monitor functional baseline.
A different app or device is affected	Restore the edition-specific metadata setting through the same control plane used to change it.	Restart and verify the recorded before-state.
DefenderRealTime is False	Check whether another antivirus is active or the PC is managed.	Do not change PUA settings until the antivirus authority is understood. [14]

APPLOCKER ENTRY GATE

Continue only when Section 5 recorded an actual return of the exact signed `LGElectronics.LGMonitorApp` package, the tested local AppLocker console and cmdlets are available, the PC is personally managed, the current Appx policy is empty or fully understood, a complete local/effective policy backup exists, and you accept the compatibility testing burden. If the package is absent, the standard path is already clean; do not reinstall it just to create a rule.

WHAT APPLOCKER IS - AND IS NOT

Microsoft documents AppLocker configuration and enforcement on current Windows 11 editions, but describes it as defense-in-depth rather than a security boundary. This runbook uses one narrow packaged-app deny, not a general application-control design. Do not adapt it by substituting another package name. [02] [03] [08]

6. Optional AppLocker A - back up, inspect, start in Audit only

This section is not part of the successful core path. It applies only after the Section 5 entry gate passes.

MANAGEMENT BOUNDARY

Current Windows 11 editions support AppLocker, but this tested local GUI path requires `secpol.msc` and the AppLocker cmdlets. Stop if either is unavailable and do not install third-party Group Policy hacks. These tools operate on local/Group Policy AppLocker, not AppLocker CSP; effective-policy output can also be incorrect for CSP deployment. Stop on MDM, domain policy or unfamiliar effective policy. [02] [15] [16]

A - EXPORT AND PARSE LOCAL AND EFFECTIVE POLICY

```
# Optional path only. Run in Windows PowerShell 5.1 as Administrator.
$Package = 'LGElectronics.LGMonitorApp'
$ExpectedPublisher = 'CN=4A5401D9-32C2-486E-A8CB-EFECB124AF89'
$ExpectedPublisherId = 'cfnzzhwkr8z5w'
$ExpectedFamily = "${Package}_${ExpectedPublisherId}"
$Types = 'Main','Framework','Resource','Bundle','Optional'
$Target = @(Get-AppxPackage -AllUsers -Name $Package `
    -PackageTypeFilter $Types -ErrorAction Stop)
if ($Target.Count -eq 0) {
    throw 'STOP: the returned tested package is not installed.'
}
if (@($Target | Where-Object {
    $_.Publisher -ne $ExpectedPublisher -or
    $_.PublisherId -ne $ExpectedPublisherId -or
    $_.PackageFamilyName -ne $ExpectedFamily
}).Count -gt 0) {
    throw 'STOP: returned signer or package family is not the tested identity.'
}
$Required = 'Get-AppLockerPolicy','Test-AppLockerPolicy'
$Missing = @($Required | Where-Object {
    -not (Get-Command $_ -ErrorAction SilentlyContinue)
})
if ($Missing.Count) { throw "STOP: missing AppLocker tools: $Missing" }
$ALRunId = '{0}-{1}' -f (Get-Date -Format 'yyyyMMdd-HHmmss'),
    ([guid]::NewGuid().ToString('N').Substring(0,8))
$ALRun = Join-Path ([Environment]::GetFolderPath('Desktop')) `
    "LG-AppLocker-$ALRunId"
New-Item -ItemType Directory -Path $ALRun -ErrorAction Stop | Out-Null
Get-AppLockerPolicy -Local -Xml -ErrorAction Stop |
    Set-Content "$ALRun\Local-Before.xml" -Encoding UTF8
Get-AppLockerPolicy -Effective -Xml -ErrorAction Stop |
    Set-Content "$ALRun\Effective-Before.xml" -Encoding UTF8
[xml](Get-Content "$ALRun\Local-Before.xml" -Raw) | Out-Null
[xml](Get-Content "$ALRun\Effective-Before.xml" -Raw) | Out-Null
Get-CimInstance Win32_Service -Filter "Name='AppIDSvc'" -ErrorAction Stop |
    Select-Object Name,State,StartMode |
    Export-Clixml "$ALRun\AppIDSvc-Before.clixml"
$ALRun
```

STEP	ACTION	PASS CONDITION
1	Open `secpol.msc` and go to Application Control Policies > AppLocker.	If the console is unavailable, stop. Do not install third-party Group Policy hacks.
2	Inspect every collection and AppLocker Properties before editing.	Existing/unfamiliar Appx rules, domain policy or CSP/MDM control: stop.
3	Under Packaged app Rules, choose Create Default Rules only when no prior Appx design exists.	Require the signed-packaged-app allow for Everyone. [19]
4	AppLocker Properties > Packaged app Rules: Configured / **Audit only**.	Do not configure EXE, MSI, Script or DLL collections.
5	Pass the Recovery Gate below, then run the service block.	Expected: State = Running and StartMode = Auto. [06]

RECOVERY GATE

Copy the complete AppLocker run folder to separate administrator-controlled storage and reopen both XML files. Policy export is not a software reinstall source. Microsoft states that AppIDSvc cannot be returned to Manual with `sc.exe` and recommends a system backup before changing its startup type. Continue only when you accept that trade-off, the preexisting policy is understood and the Packaged app collection is visibly Audit only. [06]

B - ONLY AFTER THE GATE: START AND VERIFY APPLICATION IDENTITY

```
sc.exe config appidsvc start=auto
Start-Service AppIDSvc
gpupdate /target:computer /force
Get-CimInstance Win32_Service -Filter "Name='AppIDSvc'" |
    Select-Object Name,State,StartMode
```

7. Optional AppLocker B - create the exact deny and test it

Create one package-name rule from the installed signed reference. Then predict the result before any enforcement.

STEP	WIZARD PAGE	EXACT VALUE
1	Packaged app Rules > Create New Rule	Begin only while the collection is Audit only.
2	Permissions	**Deny** / **Everyone**.
3	Condition	**Publisher**.
4	Reference	Use installed packaged app: LG Monitor App Installer.
5	Scope	Package name = `LGElectronics.LGMonitorApp`; version = all; no exceptions.
6	Name	**Block LG Monitor App Installer**.
7	Review	Keep the default signed-package allow. One narrow deny only. [05]

A - EXPORT AUDIT POLICY AND SIMULATE INSTALLED PACKAGES

```
$ALRun = '<paste exact LG-AppLocker-... folder path>'
$Package = 'LGElectronics.LGMonitorApp'
$Types = 'Main','Framework','Resource','Bundle','Optional'
$AuditStarted = Get-Date
$AuditStarted | Export-Clixml "$ALRun\Audit-Started.clixml"
gpupdate /target:computer /force
Get-AppLockerPolicy -Effective -Xml -ErrorAction Stop |
    Set-Content "$ALRun\Effective-Audit.xml" -Encoding UTF8
$Target = @(Get-AppxPackage -AllUsers -Name $Package `
    -PackageTypeFilter $Types -ErrorAction Stop)
if ($Target.Count -eq 0) { throw 'STOP: signed target package is absent.' }
$All = @(Get-AppxPackage -AllUsers -PackageTypeFilter $Types -ErrorAction Stop)
$TargetDenied = @($Target | Test-AppLockerPolicy `
    -XmlPolicy "$ALRun\Effective-Audit.xml" -User 'S-1-1-0' -Filter Denied)
$DeniedByDefault = @($All | Test-AppLockerPolicy `
    -XmlPolicy "$ALRun\Effective-Audit.xml" -User 'S-1-1-0' `
    -Filter DeniedByDefault)
[pscustomobject]@{
    TargetCount = $Target.Count
    TargetPredictedDenied = $TargetDenied.Count
    InstalledDeniedByDefault = $DeniedByDefault.Count
}
```

B - AFTER REAL USE, REVIEW WOULD-BE DENIAL EVENTS

```
$ALRun = '<paste exact LG-AppLocker-... folder path>'
$Start = Import-Clixml "$ALRun\Audit-Started.clixml"
$Logs = 'Microsoft-Windows-AppLocker/Packaged app-Execution',
        'Microsoft-Windows-AppLocker/Packaged app-Deployment'
foreach ($Log in $Logs) {
    Get-WinEvent -FilterHashtable @{
        LogName=$Log; Id=8021,8024; StartTime=$Start
    } -ErrorAction SilentlyContinue |
        Select-Object TimeCreated,Id,Message
}
```

DEFINED COMPATIBILITY SESSION

After recording AuditStarted, exercise Start, Settings, Microsoft Store, one built-in Store app, Xbox/Game Bar if used, Discord or other Store apps if applicable, Steam/Epic, mod tools and the normal monitor-control workflow. A zero-event result means little if this workload did not run. Investigate every 8021/8024 event. [07] [18]

PASS BEFORE ENFORCEMENT

TargetPredictedDenied must equal TargetCount; InstalledDeniedByDefault must be 0; the target publisher and package name must match; the default signed-package allow must remain; and no required application may have an unexplained would-be denial. Otherwise remain in Audit only.

8. Optional AppLocker C - enforce, smoke-test, then remove

Enforcement is its own decision. Do not combine the click, cleanup and verification into one opaque action.

ENFORCEMENT GATE

AppLocker enforcement applies immediately and a deny overrides an allow. If any required app is unexpectedly affected, return the Packaged app collection to Audit only before doing anything else. Do not create a broad exception to make the symptom disappear. [05] [17]

STEP	ACTION	PASS CONDITION
1	Review the Section 7 pass criteria again.	Every condition still passes; recovery XML opens.
2	AppLocker Properties > Packaged app Rules: switch only this collection to **Enforce rules**.	EXE, MSI, Script and DLL remain unchanged.
3	Run the verification block. It stops at a prompt after the policy precheck.	`AppxMode = Enabled`; target denied count equals target count; denied-by-default = 0.
4	While the block waits, smoke-test required packaged apps and attempt the LG package once; then press Enter.	No unrelated block. At least one new 8022/8025 event names the exact package and rule.
5	Only now rerun the Section 3 removal block.	Registered and provisioned counts both return 0.
6	Restart and rerun Section 4. In `secpol.msc`, recheck the exact deny, default allow and Appx Enforce mode; rerun the Section 6 service query.	Core checks pass; AppIDSvc is Running/Auto; the intended Appx rules and mode remain.

VERIFY, TEST AND CAPTURE THE ENFORCED DENY

```
$ALRun = '<paste exact LG-AppLocker-... folder path>'
$Package = 'LGElectronics.LGMonitorApp'
$RuleName = 'Block LG Monitor App Installer'
$Types = 'Main','Framework','Resource','Bundle','Optional'
gpupdate /target:computer /force
Get-AppLockerPolicy -Effective -Xml -ErrorAction Stop |
    Set-Content "$ALRun\Effective-Enforced.xml" -Encoding UTF8
[xml]$Policy = Get-Content "$ALRun\Effective-Enforced.xml" -Raw
$Appx = @($Policy.AppLockerPolicy.RuleCollection |
    Where-Object Type -eq 'Appx')
$Target = @(Get-AppxPackage -AllUsers -Name $Package `
    -PackageTypeFilter $Types -ErrorAction Stop)
$All = @(Get-AppxPackage -AllUsers -PackageTypeFilter $Types -ErrorAction Stop)
$Denied = @($Target | Test-AppLockerPolicy `
    -XmlPolicy "$ALRun\Effective-Enforced.xml" -User 'S-1-1-0' -Filter Denied)
$DefaultDenied = @($All | Test-AppLockerPolicy `
    -XmlPolicy "$ALRun\Effective-Enforced.xml" -User 'S-1-1-0' `
    -Filter DeniedByDefault)
[pscustomobject]@{
    AppxMode = $Appx.EnforcementMode
    TargetCount = $Target.Count
    TargetPredictedDenied = $Denied.Count
    InstalledDeniedByDefault = $DefaultDenied.Count
}
if ($Target.Count -eq 0 -or $Appx.Count -ne 1 -or
    $Appx.EnforcementMode -ne 'Enabled' -or
    $Denied.Count -ne $Target.Count -or $DefaultDenied.Count -ne 0) {
    throw 'STOP: enforcement precheck failed.'
}
$EnforcementStarted = Get-Date
$EnforcementStarted |
    Export-Clixml "$ALRun\Enforcement-Started.clixml"
Read-Host 'Smoke-test required apps, attempt LG once, then press Enter'
$Logs = 'Microsoft-Windows-AppLocker/Packaged app-Execution',
        'Microsoft-Windows-AppLocker/Packaged app-Deployment'
$DenyEvents = @(foreach ($Log in $Logs) {
    Get-WinEvent -FilterHashtable @{
        LogName=$Log; Id=8022,8025; StartTime=$EnforcementStarted
    } -ErrorAction SilentlyContinue
})
$TargetDenyEvents = @($DenyEvents | Where-Object {
    $_.Message -match [regex]::Escape($Package) -and
    $_.Message -match [regex]::Escape($RuleName)
})
if ($TargetDenyEvents.Count -lt 1) {
    throw 'STOP: no real target deny event was found.'
}
[pscustomobject]@{
    EnforcedTargetDenyEventCount = $TargetDenyEvents.Count
}
$TargetDenyEvents | Select-Object TimeCreated,Id,Message
```

FINAL APPLOCKER PASS

Require `EnforcedTargetDenyEventCount` of at least 1 plus a clean smoke test, not merely predicted denial. The event must name the exact package and rule. After cleanup and restart, use Section 4, the AppLocker UI and the Section 6 service query; do not rerun the target-event block after the target is removed. [07]

9. Detached option - Defender PUA protection

Stop after the core path passes. This separate, general Windows control is not LG-specific and is not required for package cleanup.

STOP CONDITIONS - READ BEFORE THE COMMAND

Skip this change when another antivirus is active, Defender is passive, policy is organization-managed, Tamper Protection prevents the write or compatibility is unclear. Do not add broad game-library or application exclusions as a substitute for investigating a specific detection. [13] [14]

CHECK FIRST; NEVER WEAKEN AN ALREADY ENABLED STATE

```
$RunDir = '<paste exact LG-Run-... folder path>'
$AuditStartPath = Join-Path $RunDir 'PUA-Audit-Started.clixml'
$Status = Get-MpComputerStatus
$Before = (Get-MpPreference).PUAProtection
$Status | Select-Object AMRunningMode,AntivirusEnabled,
    RealTimeProtectionEnabled
[pscustomobject]@{ PUAProtection = $Before }
# Decision:
# 1: keep blocking; 2: keep Audit; 0: stage Audit only if active/unmanaged.
# Preserve the first audit-start timestamp across reruns.
if ($Before -in 0,2 -and
    -not (Test-Path -LiteralPath $AuditStartPath)) {
    [pscustomobject]@{ Captured=Get-Date; PUAProtection=[int]$Before } |
        Export-Clixml $AuditStartPath
}
if ($Before -eq 0 -and $Status.AMRunningMode -eq 'Normal' -and
    $Status.AntivirusEnabled -and $Status.RealTimeProtectionEnabled) {
    Set-MpPreference -PUAProtection AuditMode
}
(Get-MpPreference).PUAProtection
```

VALUE	MEANING	ACTION
0	Disabled	If Defender is active and unmanaged, Audit mode is the cautious first change.
1	Enabled / blocking	Pass. Keep it. Do not lower it to reproduce this guide.
2	Audit mode	Keep it while testing. Review only events created after the recorded audit start.

IF VALUE IS 2, TEST NORMAL USE, THEN REVIEW ONLY NEW AUDIT EVENTS

```
$RunDir = '<paste exact LG-Run-... folder path>'
$AuditStartPath = Join-Path $RunDir 'PUA-Audit-Started.clixml'
if (-not (Test-Path -LiteralPath $AuditStartPath)) {
    throw 'STOP: the audit-start record is missing.'
}
$Start = (Import-Clixml $AuditStartPath).Captured
if ($null -eq $Start) { throw 'STOP: invalid audit-start record.' }
Get-WinEvent -FilterHashtable @{
    LogName='Microsoft-Windows-Windows Defender/Operational'
    Id=1160
    StartTime=$Start
} -ErrorAction SilentlyContinue |
    Select-Object TimeCreated,Id,Message
```

ONLY AFTER A CLEAN AUDIT: ENABLE, UPDATE, SCAN AND VERIFY

```
Set-MpPreference -PUAProtection Enabled
Update-MpSignature
$ScanRequested = Get-Date
Start-MpScan -ScanType QuickScan
$Status = Get-MpComputerStatus
[pscustomobject]@{
    PUAProtection = (Get-MpPreference).PUAProtection
    QuickScanEndTime = $Status.QuickScanEndTime
    FreshScanCompleted = $Status.QuickScanEndTime -ge $ScanRequested
}
```

Expected result, restart and recovery

`PUAProtection = 1` and `FreshScanCompleted = True`. No restart is normally required; follow any Windows Security prompt. If either result fails, remain in Audit mode or restore the recorded prior value in Rollback. Smart App Control is separate; this guide neither requires nor changes it.

10. Rollback - reverse the smallest layer first

Rollback should restore the recorded before-state, not merely make an error disappear. Removed applications require reinstallation.

PROBLEM	FIRST RECOVERY ACTION	PASS RESULT
Unexpected AppLocker block	Immediately set Packaged app Rules back to **Audit only**, refresh Group Policy, then delete only **Block LG Monitor App Installer** and refresh again.	Required apps run; exact LG deny is absent. Keep AppIDSvc running while troubleshooting. [20]
AppLocker baseline must be restored	Prefer restoring the guide-owned Appx collection, rule and mode manually. Use full local-policy replacement only as the last resort below. Do not try to return AppIDSvc to Manual with sc.exe.	Local policy matches the captured before-state; any retained Automatic service state is documented. Never import Effective-Before.xml. [06]
Metadata side effect	Restore the prior Local Group Policy state through Local Group Policy, refresh policy and verify.	The recorded setting and affected peripheral workflow return.
PUA compatibility issue	Run the exact-state restore block. Do not create a broad exclusion.	PUAProtection equals the value recorded before the guide.
Needed LG function was removed	Reinstall only the intended application from the recorded official LG source.	Firmware/settings access works; package state is re-inventoried. [23]

FULL-POLICY REPLACEMENT GATE - READ BEFORE THE COMMAND

Do not use the emergency command on a managed PC, for CSP/MDM policy, after unrelated policy changes or with an unverified backup. It replaces local policy; it is not a merge. Review `-WhatIf` output and the newly exported current policy before typing the exact confirmation. Reopen both after-state XML files, compare intended collections, rules and modes, then restart and rerun the relevant functional checks. [16]

LAST RESORT: FULL LOCAL APPLOCKER RESTORE

```
# LAST RESORT: local/unmanaged AppLocker only.
# Saved local XML must be known-good; no later AppLocker changes.
$ALRun = '<paste exact LG-AppLocker-... folder path>'
$Backup = Join-Path $ALRun 'Local-Before.xml'
[xml](Get-Content -LiteralPath $Backup -Raw -ErrorAction Stop) | Out-Null
$Stamp = Get-Date -Format 'yyyyMMdd-HHmmss'
$Current = Join-Path $ALRun "Local-Before-Emergency-$Stamp.xml"
Get-AppLockerPolicy -Local -Xml -ErrorAction Stop |
    Set-Content $Current -Encoding UTF8
[xml](Get-Content $Current -Raw -ErrorAction Stop) | Out-Null
Set-AppLockerPolicy -XmlPolicy $Backup -WhatIf
$Confirm = Read-Host 'Type REPLACE LOCAL APPLOCKER POLICY to continue'
if ($Confirm -cne 'REPLACE LOCAL APPLOCKER POLICY') {
    throw 'STOP: full-policy replacement was not confirmed.'
}
Set-AppLockerPolicy -XmlPolicy $Backup -ErrorAction Stop
gpupdate /target:computer /force
Get-AppLockerPolicy -Local -Xml -ErrorAction Stop |
    Set-Content "$ALRun\Local-After-Restore.xml" -Encoding UTF8
Get-AppLockerPolicy -Effective -Xml -ErrorAction Stop |
    Set-Content "$ALRun\Effective-After-Restore.xml" -Encoding UTF8
```

RESTORE THE EXACT PRIOR DEFENDER PUA VALUE

```
$RunDir = '<paste exact LG-Run-... folder path>'
$Before = Import-Clixml "$RunDir\Defender-Before.clixml"
Set-MpPreference -PUAProtection ([int]$Before.PUAProtection)
(Get-MpPreference).PUAProtection
```

11. Authoritative references

Microsoft and LG primary documentation reviewed for guide version 1.0.0 on 29 July 2026. Current vendor documentation takes precedence if it changes.

#	SOURCE	URL
01	Prevent device metadata retrieval from the network	Open official source
02	AppLocker requirements and supported Windows editions	Open official source
03	KB5024351: removal of Windows edition checks for AppLocker	Open official source
04	Create an AppLocker rule for packaged apps	Open official source
05	AppLocker allow and deny actions	Open official source
06	Configure the Application Identity service	Open official source
07	Use Event Viewer with AppLocker	Open official source
08	AppLocker overview and defense-in-depth guidance	Open official source
09	Get-AppxPackage	Open official source
10	Remove-AppxPackage	Open official source
11	Get-AppxProvisionedPackage	Open official source
12	Remove-AppxProvisionedPackage	Open official source
13	Detect and block potentially unwanted applications	Open official source
14	Microsoft Defender Antivirus compatibility modes	Open official source
15	Get-AppLockerPolicy	Open official source
16	Set-AppLockerPolicy	Open official source
17	Deploy AppLocker policies by using Enforce rules	Open official source
18	Test-AppLockerPolicy	Open official source
19	Create AppLocker default rules	Open official source
20	Delete an AppLocker rule	Open official source
21	Automatic installation for UWP device apps	Open official source
22	INF AddSoftware directive	Open official source
23	LG product support, manuals, software and drivers	Open official source
24	LG Monitor OnScreen Control: update monitor software	Open official source
25	Microsoft Store: LG Monitor App Installer	Open official source
26	Driver Package Container Metadata	Open official source
27	System Configuration tools in Windows	Open official source